자바 핵심 학습 내용 총정리

핵심적인 자바 프로그래밍 언어의 문법, 객체지향 개념, I/O, 주요 클래스 및 실무 아키텍처 패턴 체계적 정리

---

1. 변수, 자료형, 제네릭 (ch01)

변수와 메모리 구조

- **변수와 상수(final):** 재할당 가능한 변수와 달리 `final` 키워드가 선언된 상수는 초기화 이후 값을 변경할 수 없어 프로그램의 안정성을 높입니다.
- **클래스 기반 객체 생성:** 사용자 정의 타입을 정의하고 `new` 연산자를 통해 힙(Heap) 메모리에 인스턴스를 생성하여 참조합니다.

타입 시스템과 제네릭

- **Object 타입의 한계:** 모든 객체를 상위 타입으로 수용할 수 있으나, 데이터 인출 시 강제 형변환이 필요하며 타입 안정성이 떨어지는 단점이 존재합니다.
- **제네릭(Generics):** 클래스나 메서드를 정의할 때 타입을 확정하지 않고, 인스턴스 생성 시점에 타입을 지정하여 컴파일 타임에 강력한 타입 체크를 수행합니다.
    - **다이아몬드 연산자 (`<>`):** 인스턴스 생성 시 중복된 타입 파라미터를 생략할 수 있는 간결한 표기법을 제공합니다.
    - **와일드카드 (`?`):** `? extends T` 또는 `? super T`를 통해 상한/하한 경계를 설정하여 유연한 구조의 참조를 지원합니다.

---

2. 메서드와 캡슐화 (ch02, ch03)

메서드 설계와 접근 제어

- **메서드의 목적:** 중복 코드를 제거하고, 로직을 모듈화하여 코드의 재사용성과 가독성을 극대화합니다.
- **반환형과 매개변수:** 반환값이 없는 `void`와 명시적 반환 타입을 구분하며, 메서드 선언부의 매개변수(Parameter)와 실제 호출 시 전달되는 인자(Argument)를 명확히 구분합니다.

접근 제어자	클래스 내부	동일 패키지	자식 클래스	전체 접근	비고
`private`	O	X	X	X	정보 은닉 및 안전한 캡슐화
`default`	O	O	X	X	패키지 레벨 접근 제한
`protected`	O	O	O	X	상속 관계 접근 허용
`public`	O	O	O	O	어디서나 접근 가능

캡슐화와 멤버 키워드

- **캡슐화 (Encapsulation):** 필드를 `private`으로 제한하고 안전한 Getter / Setter 패턴을 통해 객체의 내부 상태 변경을 통제합니다.
- **`this` 키워드:** 인스턴스 자신을 가리키는 참조 변수로, 메서드 파라미터와 인스턴스 멤버 변수의 이름이 같을 때 이를 명확히 구분합니다.
- **인스턴스 멤버 vs 정적 멤버 (static):**
  - **인스턴스 멤버:** `new` 연산으로 힙 메모리에 생성되며 객체마다 독립된 상태를 유지합니다.
  - **정적 멤버 (`static`):** 클래스 로딩 시 메서드 영역(Method Area)에 생성되어 모든 인스턴스가 전역으로 공간을 공유합니다.

---

3. 배열과 제어문, I/O (ch04, ch05)

배열 및 동적 배열 구조

- **배열 (Array):** 연속된 메모리 공간에 고정 크기로 할당되는 참조 자료형입니다.
- **객체 배열:** 객체 자체가 아닌 객체의 참조 주소를 저장하는 배열로, 각 인덱스마다 별도의 `new` 인스턴스 생성이 필요합니다.
- **동적 배열 확장 메커니즘:** 크기가 고정된 배열의 한계를 극복하기 위해, 내부 배열이 찼을 때 더 큰 배열(기존 크기 + α)을 새로 할당하고 데이터를 복사하는 방식입니다 (`ArrayList` 내부 동작 원리).

제어문과 흐름 제어

- **`switch ~ case`:** 일치하는 케이스 조건에 맞춰 실행 흐름을 제어하며, 의도하지 않은 조건 누수를 막기 위해 `break`를 명시합니다.
- **반복문 (`for` / `while`):** 조건에 따른 반복 수행 및 중첩 반복 알고리즘 처리 시 활용됩니다.

- **`continue` & `break`:** `continue`는 현재 루프의 남은 코드를 스킵하고 다음 반복으로 넘어가며, `break`는 반복문을 즉시 탈출합니다.

입출력 (I/O) 처리

- **`Scanner`:** 콘솔을 통한 기본 입력 및 한 줄 단위 텍스트 읽기를 제공합니다.
- **`BufferedReader` + `FileReader`:** 버퍼링을 통해 파일 입출력 성능을 대폭 향상시키며, EOF(`null`) 도달 시까지 안정적으로 한 줄씩 읽어옵니다.
- **`StringBuilder`:** 가변(Mutable) 문자열 객체로, 잦은 문자열 결합 연산 시 발생하는 불필요한 객체 생성을 막아 메모리를 절약합니다.

---

4. 객체지향 심화: 상속, 다형성, 추상화 (ch06, ch07)

다형성과 객체 재정의

- **오버로딩 (Overloading):** 동일한 이름의 메서드/생성자에 매개변수의 타입, 개수, 순서를 다르게 정의하여 유연한 인터페이스를 제공합니다.
- **상속과 `super`:** 자식 객체 생성 시 부모 생성자가 먼저 호출되며, `super` 키워드를 사용해 부모 클래스의 멤버에 접근합니다.
- **다형성 (Polymorphism):**
  - **업캐스팅 (Upcasting):** 자식 객체를 부모 타입 참조 변수로 관리하여 상위 규격으로 일괄 처리가 가능합니다.
  - **오버라이딩 (Overriding):** 하위 타입에서 메서드를 재정의할 경우, 부모 타입으로 참조하더라도 실제 런타임 인스턴스의 오버라이딩된 메서드가 우선 실행됩니다.
  - **다운캐스팅 & `instanceof`:** 업캐스팅된 참조를 하위 고유 타입으로 복원할 때, 런타임 `ClassCastException` 방지를 위해 `instanceof` 검증을 수행합니다.

추상화 및 컬렉션 인터페이스

구분	추상 클래스 (abstract class)	인터페이스 (interface)
목적	미완성 메서드를 포함하며 상속 관계의 가이드 제공	객체가 준수해야 하는 규격 및 스펙 정의
상속/구현	단일 상속 (`extends`)	다중 구현 (`implements`)
필드 선언	일반 인스턴스 변수 및 상수 소유 가능	`public static final` 상수만 가능

구분	추상 클래스 (abstract class)	인터페이스 (interface)
핵심 가치	코드 재사용 및 클래스 간 연관 관계 형성	다형성 극대화 및 계층 간 결합도 완화

- **`List` 인터페이스:** `ArrayList`, `LinkedList` 등의 구현체를 단일 `List` 규격으로 다형적 접근하여 코드 유연성을 극대화합니다.

---

5. Object 클래스와 동등성 규약 (ch08)

최상위 Object 클래스

- **최상위 클래스:** 자바의 모든 클래스는 암묵적으로 `Object` 클래스를 상속받습니다.
- **`getClass()`:** 실행 중인 인스턴스의 런타임 클래스 메타정보를 확인합니다.

동일성과 동등성

[동일성 (Identity)] → == 연산자 → 메모리 참조 주소 비교

[동등성 (Equality)] → equals() → 객체 내부 데이터 값 비교

- **동등성 재정의:** 데이터 값 비교를 위해 `equals()`를 재정의하며, `Objects.equals()`를 활용해 안전한 `null` 처리를 수행합니다.
- **`hashCode()`와의 계약:** `equals()` 결과가 `true`인 두 객체는 반드시 동일한 `hashCode()` 값을 반환해야 합니다. (HashSet, HashMap의 오퍼레이션 정상 작동 보장)
- **`toString()`:** 객체의 내부 데이터 상태를 사람이 읽기 쉬운 문자열 형태로 시각화합니다.

---

6. 디자인 패턴, 롬복, 실무 아키텍처 (ch10, ch11, ch12)

디자인 패턴 및 롬복 (Lombok)

- **싱글톤 패턴 (Singleton Pattern):** 애플리케이션 실행 중 단 하나의 인스턴스만 존재하도록 보장합니다.
  - `private` 생성자 + `private static` 인스턴스 변수 + `public static getInstance()` 조합.
- **롬복 (Lombok):** 어노테이션 기반으로 보일러플레이트 코드를 자동 생성합니다.

    - `@Getter`, `@Setter`, `@ToString`, `@EqualsAndHashCode`
    - `@NoArgsConstructor`, `@AllArgsConstructor`, `@RequiredArgsConstructor`
    - `@Data`: 위 핵심 어노테이션들을 포함하는 종합 자동화 어노테이션.

자료구조 내부 구현 비교

- **`ArrayList`:** 연속된 배열 기반 구조로 인덱스를 통한 빠른 조회가 가능하나, 크기 확장이나 중간 삽입/삭제 시 배열 복사 비용이 크게 발생합니다.
- **`LinkedList`:** 노드(`item` + `next` 참조 주소)의 연결 구조로 포인터 변경을 통한 삽입/삭제가 용이하나, 요소 접근 시 순차 탐색이 필요합니다.

계층형 아키텍처 (Layered Architecture)

콘솔 애플리케이션 실습을 위한 계층 분리 모범 사례:

- **Entity (`Todo`):** 비즈니스 도메인의 데이터 모델을 표현.
- **Repository (`Repository<T>`, `TodoRepository`):** 싱글톤 기반의 인메모리 데이터 저장소로, CRUD 추상화 및 `List.copyOf()`를 활용한 불변성 데이터 반환 보장.
- **View (`View`, `HomeView`, `RegisterView`, `ListView`):** 사용자 입출력 콘솔 UI 및 화면간 라우팅 로직을 독립적으로 분리.